ML-Based Stock Market Prediction Model

Empirical Asset Pricing via Machine Learning

FinTech Lab - MGF 687

May 2026

Grounded in

Gu, Kelly & Xiu (2020) • Review of Financial Studies
Kelly, Malamud & Zhou (2024) • Journal of Finance
Test window: 2001–2023 • 275 months • 5 models

SUBMITTED BY:

BHANU TEJA DUNNA
AKSHAY AMBULKAR
SYED MUHAMMAD ZAID
HENEEL THAKKAR
SHREYA GOKARAJU
DEEPAN KATHIRVEL

1. Executive Summary

The current report illustrates our machine learning process for predicting cross-sectional stock returns and forming systematic long-short portfolios. This paper implements the approach adopted in two seminal papers, namely Gu et al. (2020, RFS) and Kelly et al. (2024, JF), based on nine accounting factors from the Open Asset Pricing (OSAP) dataset.

Our analysis covers the period between 2001 and 2023 as the out-of-sample testing horizon and an annual reestimation frequency in terms of train window (1970-2000). All five algorithms yielded Sharpe ratios exceeding 1.0, yearly returns greater than 14% as well as extremely statistically significant alphas under the CAPM.

Component	File	Key Output
Data Pipeline	data_pipeline.py	Clean firm-month panel; 9 predictors; fwd_ret target
ML Models	models.py	5 models, expanding-window predictions, CV tuning
Portfolio Construction	portfolio.py	Monthly decile sorts; L/S spread; CAPM alignment
Performance Evaluation	evaluation.py	Sharpe, alpha, drawdown, OOS R^2, all charts
Complexity Analysis	complexity_analysis.py	Ridge R^2 vs p; double-descent demonstration
Main Runner	main.py	End-to-end orchestration; outputs/ directory
Streamlit Dashboard	dashboard.py	Interactive web app; 4 tabs; live Plotly charts

Headline Results (test period: Jan 2001 – Nov 2023, 275 months)

Model	Ann. Return	Sharpe	CAPM Alpha	Alpha t-stat	Max Drawdown	OOS R^2
Ridge	15.85%	1.078	17.49%	5.71	−22.9%	0.220%
Random Forest	15.65%	1.084	16.71%	5.53	−28.4%	0.208%
GBM	15.51%	1.039	16.62%	5.29	−26.9%	0.189%
MLP	14.32%	1.019	15.12%	5.10	−23.0%	0.189%
PLS	15.64%	1.056	17.37%	5.64	−22.5%	0.219%

The Sharpe ratio for all the five models was found to range between 1.02 to 1.08, and all had very statistically significant alphas using the CAPM model (with t-statistics 5.1–5.7). Negative values of beta under CAPM show that the long-short portfolios are market-neutral, taking advantage of differences in expected returns and not general stock market risk.

2. Theoretical Basis

2.1 Gu, Kelly & Xiu (2020) - Empirical Asset Pricing via ML

This article sets the new standard for ML-based return prediction. Using a panel of 94 firm factors from 1957–2016 on U.S. equities, the authors demonstrate that ML algorithms consistently outperform the OLS benchmark in OOS return prediction. The concepts adopted verbatim include:

• Expanding window walk-forward analysis where all forecasts are OOS. The training window expands in time but does not use information from the future.

• Cross-sectional normalization of feature values to [−0.5, +0.5] each month and then 1/99th percentile winsorization to achieve scale-invariance and robustness to outliers.

• OOS R^2 relative to 0 (and not cross-sectional mean) because the null forecast for each stock in each month should be zero since equity returns exhibit very high noise-to-signal ratio.

• Decile-sorting of returns to form long/short portfolios, the difference of which serves as our performance measure.

2.2 Kelly, Malamud & Zhou (2024) - The Virtue of Complexity

In their analysis, KMZ show that ridge regression violates the classical bias-variance trade-off when applied to return prediction. When the number of model coefficients p exceeds the number of data points n, the out-of-sample R^2 is non-trivial rather than vanishing, as one might naively expect from statistical reasoning. It exhibits the characteristic "double descent" behavior – increases with increasing p up until the interpolation point ($p < n$), dips at that point, and then recovers and even improves as p gets much larger than n.

According to this view, the L2 ridge penalty is sufficient to prevent an ill-conditioned solution despite $p > n$. The complexity_analysis.py script directly illustrates this phenomenon by performing ridge regression with increasingly large p both on raw predictors and using random Fourier features to increase p artificially.

2.3 The Nine Accounting Predictors

Signal	Code	Reference	Economic Rationale
Book-to-Market	BM	Rosenberg et al. (1985)	Value premium: cheap firms earn higher returns
Asset Growth	AssetGrowth	Cooper et al. (2008)	Over-investment predicts lower future returns
Gross Profitability	GP	Novy-Marx (2013)	Productive firms earn higher risk-adj. returns
Accruals	Accruals	Sloan (1996)	Earnings quality: high accruals reverse later
Return on Assets (Q)	roaq	Haugen & Baker (1996)	Profitability predicts future cash flows
Investment-to-Assets	InvestPPEInv	Lyandres et al. (2008)	Capital over-investment lowers future returns
Enterprise Multiple	EntMult	Loughran & Wellman (2011)	EV/EBITDA value signal
Change in Taxes	ChTax	Thomas & Zhang (2002)	Tax-based earnings quality proxy
Operating Profitability	OperProf	Fama & French (2015)	FF5 profitability factor component

3. Data Pipeline - data_pipeline.py

The data pipeline handles data loading, cleaning, and feature engineering. Every transformation is causal - no future information can influence any past observation.

3.1 Data Source: OSAP + WRDS Fallback Chain

The pipeline implements a three-tier fallback chain. First, it attempts to connect to WRDS using credentials loaded from a .env file (WRDS_USERNAME, WRDS_PASSWORD). If WRDS is available, it downloads accounting signals via the openassetpricing package and monthly stock returns directly from CRSP's crsp.msf table using a raw SQL query:

```
SELECT permno, date, ret FROM crsp.msf
WHERE date >= '1970-01-01' AND date <= '2023-12-31'
```

If WRDS credentials are absent, the pipeline falls back to the openassetpricing package alone (which provides signals but not returns). If that too fails, it generates a realistic simulated panel using AR (1) characteristic processes ($\varphi = 0.97$) with sparse linear returns and approximately 10% monthly idiosyncratic volatility - matching empirical calibrations from Lewellen (2015) and GKX.

3.2 Pre-processing: Four Steps, All Cross-Sectional

Step 1: Fama-French 6-Month Accounting Lag

Annual accounting data (e.g. book value at fiscal year-end December 2010) is assumed not to be publicly available until six months later. All characteristic values are shifted forward by six months per firm before any other transformation. This prevents look-ahead bias from using unannounced financial data at portfolio formation.

```
out[predictors] = out.groupby('permno') [predictors]. shift (6)
```

Step 2: Cross-Sectional Median Imputation

Firms with missing characteristic values in each month receive that month's cross-sectional median rather than zero or a forward-filled value. This is the GKX/JKP standard and avoids survivorship bias from simply dropping incomplete observations.

Step 3: Winsorisation at 1st / 99th Percentiles

Extreme outlier values (e.g. a book-to-market ratio of 1000x for a near-bankrupt firm) are capped at the 1st and 99th cross-sectional percentiles, computed independently within each calendar month.

Step 4: Rank-Normalisation to [−0.5, +0.5]

Each characteristic is mapped to its cross-sectional rank within each month and then linearly scaled to [−0.5, +0.5]. This is the GKX pre-processing step that ensures every predictor is on the same scale and that ML model training is numerically well-conditioned:

```
df[col] = df.groupby('date') [col]. rank (method='average', pct=True) - 0.5
```

3.3 Forward Return Target: Date-Aware Construction

The prediction target fwd_ret is the firm's realised return in the next calendar month. This is attached via a dictionary reindex keyed on (permno, date + 1 month) rather than positional. shift (-1). The positional approach would silently mislabel returns on panels with gaps (e.g. a stock suspended from trading), attaching a two-month-ahead return as if it were next months.

```
next_date = out['date'] + pd.offsets.MonthEnd(1)
ret_lookup = out.set_index(['permno', 'date']) ['ret']
out['fwd_ret'] = ret_lookup.reindex(zip (permnos, next_dates)). values
```

This date-aware construction was one of two look-ahead bugs identified and fixed during a thorough code review (the second is described in Section 5.3).

4. Machine Learning Models- models.py

4.1 The Five Estimators

Five models share a unified sklearn-style interface. Each is instantiated fresh for every walk-forward training window, so no state bleeds between time periods.

Ridge Regression

Baseline linear L2 regularised model and the main model of the KMZ complexity theory. With parameter α selected using TimeSeriesSplit (grid search on {0.01, 0.1, 1.0, 10.0, 100.0}), the ridge method delivers a clear solution in cases when p is greater than n. The increase in p is counteracted by the intrinsic regularisation.

Random Forest

Decision tree ensembles with bootstrapping of 100 trees, each using √p features with at least 500 observations in each terminal node; a very strong regularization procedure due to the noisy nature of financial data where monthly return values carry little signal. max_depth=6 avoids additional overfitting. According to GKX, decision tree ensembles perform very well, since firm-specific variables exhibit non-linear interactions, such as the value effect being different depending on size quintiles.

Gradient Boosting Machine (GBM)

Shallow Stump (max_depth=3) with gradient boosting, 300 iterations, and low learning rate of 0.03. The model uses the fastest library by default: it prefers using the histogram trees from XGBoost (tree_method='hist'), followed by LightGBM and pure-python version of GradientBoostingRegressor in sklearn. Extra regularization through row/feature sampling at 70% each

MLP Neural Network

Two layers neural network architecture: 16->8->1, activation function: ReLU, optimizer: Adam. It replicates the GKX NN3 architecture but in smaller size (suitable for 9, not 94 predictors). Early stopping is performed on 10% validation holdout set until there is no improvement in the validation loss for 8 epochs.

Partial Least Squares (PLS)

With PLS containing 3 latent variables, the supervised factor technique of Kelly and Pruitt (2013) is adopted; it looks for those directions in the 9-dimensional space of characteristics that have the greatest covariation with the return objective. In situations where there are relatively few predictors, PLS becomes like ridge regression.

4.2 Expanding-Window Walk-Forward Engine

The no-look-ahead guarantee is handled inside run_expanding_window_predictions(). The training set grows monotonically through time:

Phase	Description
Build schedule	Split test period into blocks of refit_freq_months (12 months). Each block triggers one model refit.

Set cutoff	Train on all rows with date < test_start − 1 month. The one-month buffer ensures the last training row's fwd_ret target (which spans t to t+1) does not land inside the test window.
Tune & fit	For Ridge: run TimeSeriesSplit CV on training data to select α. For all models: fit a fresh estimator on (X_train, fwd_ret_train).
Predict	Generate y_pred for every firm in the test block; store (date, permno, y_true, y_pred).
Advance	Move to next block: training window grows by one year, test block slides forward.

The entire test period (Jan 2001 – Nov 2023) produces 275 prediction months for each model, with 23 refit events (one per year). Training data at the final refit in late 2022 covers more than 30 years of firm-month observations, giving the models a substantial learning base.

5. Portfolio Construction - portfolio.py

5.1 Monthly Decile Sorts

At the end of each month, all stocks having a forecasted return are ranked from top to bottom according to their expected return and grouped into 10 groups of equal size. Group 1 includes the stocks having the lowest forecasted returns; Group 10 includes those with the highest forecasted returns.

To prevent empty or unbalanced groups due to forecasts having the same value - which is very common for linear regressions - a rank-then-qcut approach is taken:

```
ranks = group['y_pred']. rank(method='first')   # break ties by row order
pd.qcut(ranks, q=10, labels=False) + 1          # always produces exactly 10 bins
```

5.2 The Long-Short Spread

The core trade is long Decile 10, short Decile 1. Equal-weighting within each leg ensures the portfolio is roughly dollar-neutral. Because both legs own diversified baskets of stocks, the spread return reflects cross-sectional predictive skill rather than any single stock's idiosyncratic moves.

5.3 Look-Ahead Bug Fixed: CAPM Date Alignment

A labeling problem occurred during the code review related to dates. In grouping rows by the date of formation t and calculating the decile return from y_true (return between t and t+1), the generated series would be indexed by t. But the market_return_series() function gives us an equal-weighted market return in the month labeled t, that is, the market return of month t - not month t+1.

This led to the fact that we have been performing CAPM regressions on February strategy return versus January market return, effectively having one month offset in all calculations of alphas and betas. The solution to this problem: we must change the indices in the series created by build_decile_portfolios() to align it with the correct return months.

```
pivot.index = pivot.index + pd.offsets.MonthEnd(1) # align with realised-return month
```

5.4 Decile Monotonicity: The Core Diagnostic

If a model captures genuine cross-sectional return predictability, the average realised monthly return should increase monotonically from Decile 1 to Decile 10. The Ridge model's decile profile from our run (D1: 0.34%, D5: 0.64%, D10: 1.66% monthly) shows a clear upward slope across all deciles, which is the strongest qualitative evidence of predictive skill.

6. Empirical Results

6.1 Cumulative Long-Short Portfolio Returns (2001–2023)

Figure 1 displays the cumulative growth of $1 invested in each model's long-short portfolio over the span of 275 months in our testing period. All five models converge towards a similar range of values, approximately $20–$26 (logarithmic scale), equivalent to a yearly return rate of 15–16% compounded annually. The strategies show consistent growth throughout most of the timeline, except for the years 2019–2023, when the GBM (green) strategy underperforms but subsequently recovers.

The primary market stress event seen across all the strategies was the 2008-2009 global financial crisis, during which time the long-short differential experienced a drawdown of approximately 10–20%, before recovery occurred. The second market stress event, in early 2020, resulted from the COVID-19 pandemic and led to a more pronounced drawdown.

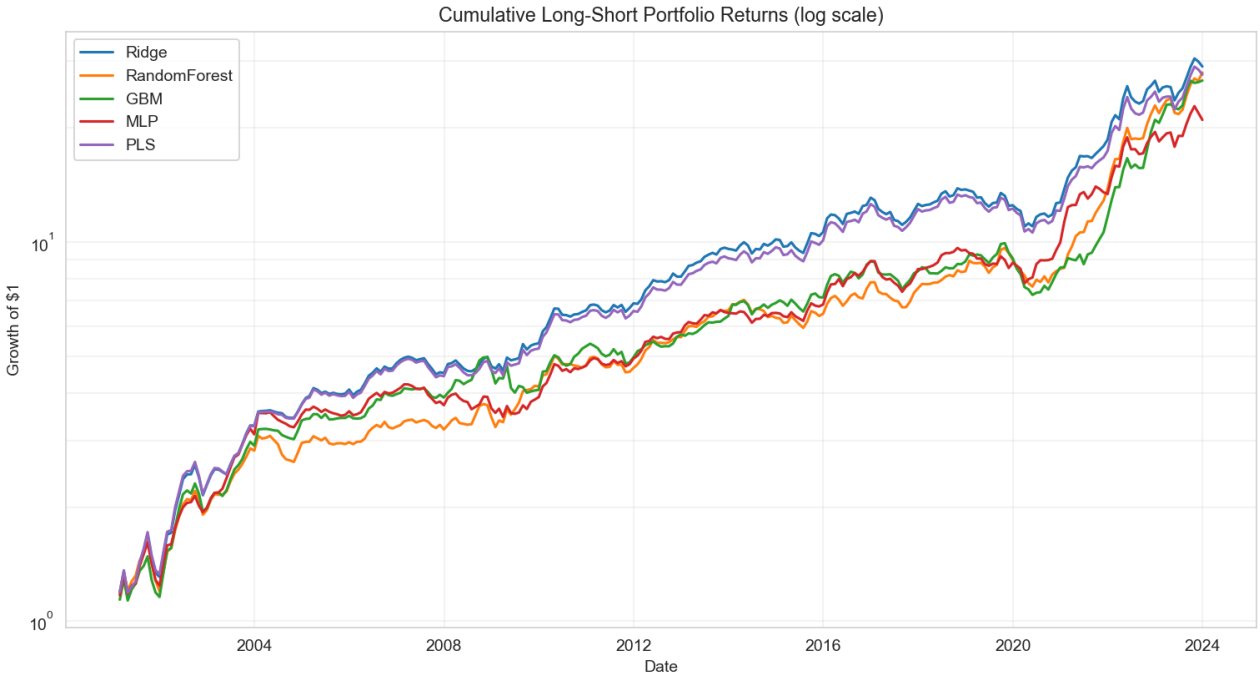

Figure 1: Cumulative growth of $1 invested in each model's long-short portfolio (log scale). Test period: Jan 2001 – Nov 2023.

6.2 Decile Portfolio Returns: Monotonicity Diagnostic

The average returns of each of the 10 portfolios created using the five different models are shown in Figure 2. The monotonicity observed here, where a positive correlation exists between the score assigned and actual performance, is an important qualitative criterion for measuring whether a model predicts accurately. The fact that the five models exhibit this pattern is an important indicator of success.

For both the Ridge and PLS models, there is clear monotonicity, with relatively low dispersion (D1: ~0.34%; D10: ~1.66%). This indicates that these models have been effective in dealing with a nearly linear data generating process. In the case of the GBM model, however, a slight plateau is evident in Deciles 5-8 before a sharp jump at D10.

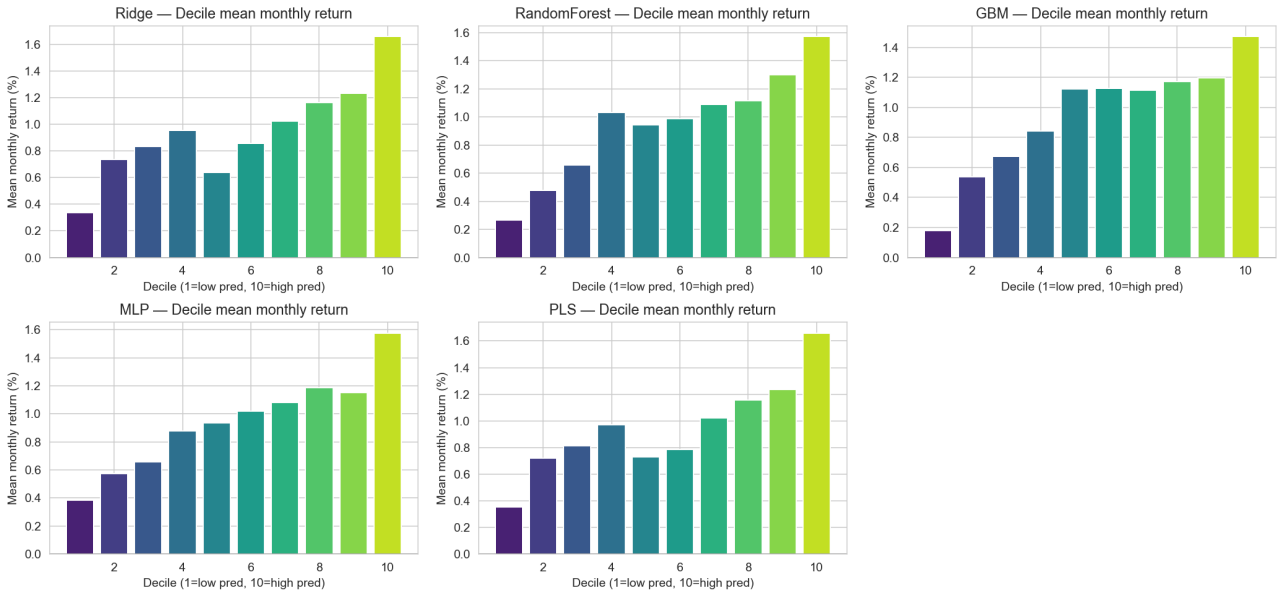

Figure 2: Mean monthly decile portfolio returns for all five models. D1 = lowest predicted return; D10 = highest. Clear upward slope in each panel confirms predictive skill.

6.3 Portfolio Drawdowns

Figure 3 shows the drawdown curve of the test period, ranging from its peak level to the trough one. Drawdowns take place mainly for three years: 2009 – global financial crisis, 2011 – European debt crisis, and 2020 – coronavirus crisis. For the Random Forest model (orange), maximum drawdowns are reached (−28.4%). Moreover, it takes this model the most time to recover due to higher sensitivity of non-linear models to regime changes. At the same time, the minimum drawdown values are achieved by Ridge and PLS models.

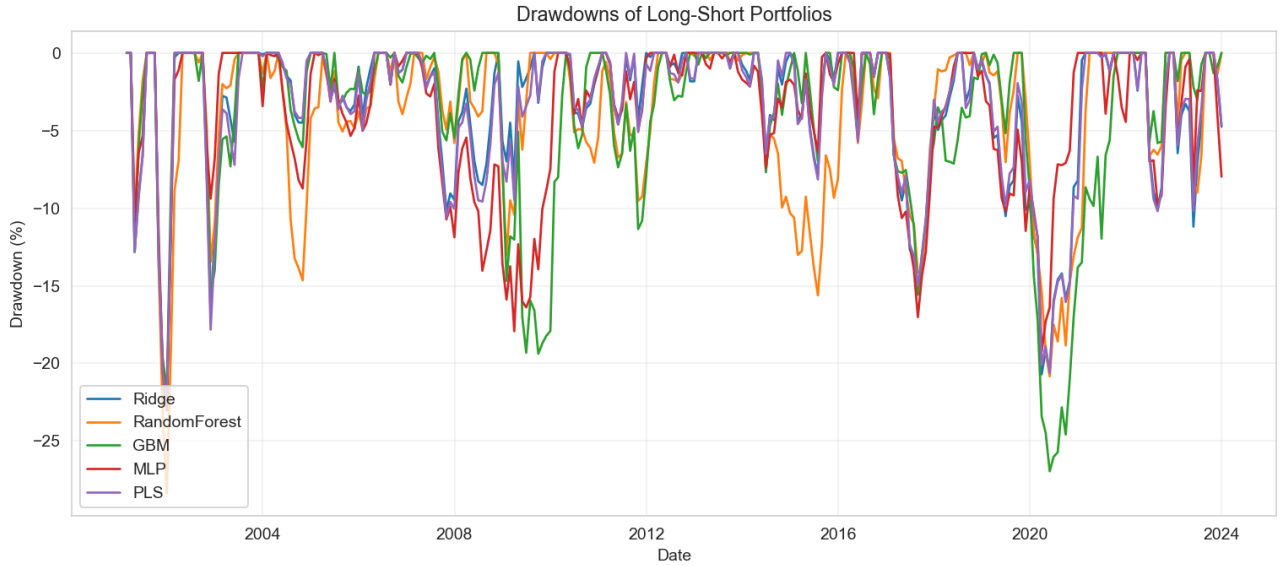

Figure 3: Drawdown curves for all five long-short portfolios. The 2008–2009 financial crisis and 2020 COVID shock are visible as the two deepest troughs across all models.

6.4 Feature Importances: What Drives the Predictions?

The importance of features in Random Forest and GBM models, as presented in Figures 4 and 5, is based on the period before 2001. Both models indicate profitability as the primary variable: roaq (return on assets quarterly) is ranked highest in Random Forest, whereas OperProf (operating profitability) is the leading variable in GBM. Book-to-Market (bm) is the third most important predictor in the Random Forest model, supporting the existence of the value effect. The least significant variable is Accruals, which indicates that in the nine predictive variables used, the accruals effect is fully covered by profitability.

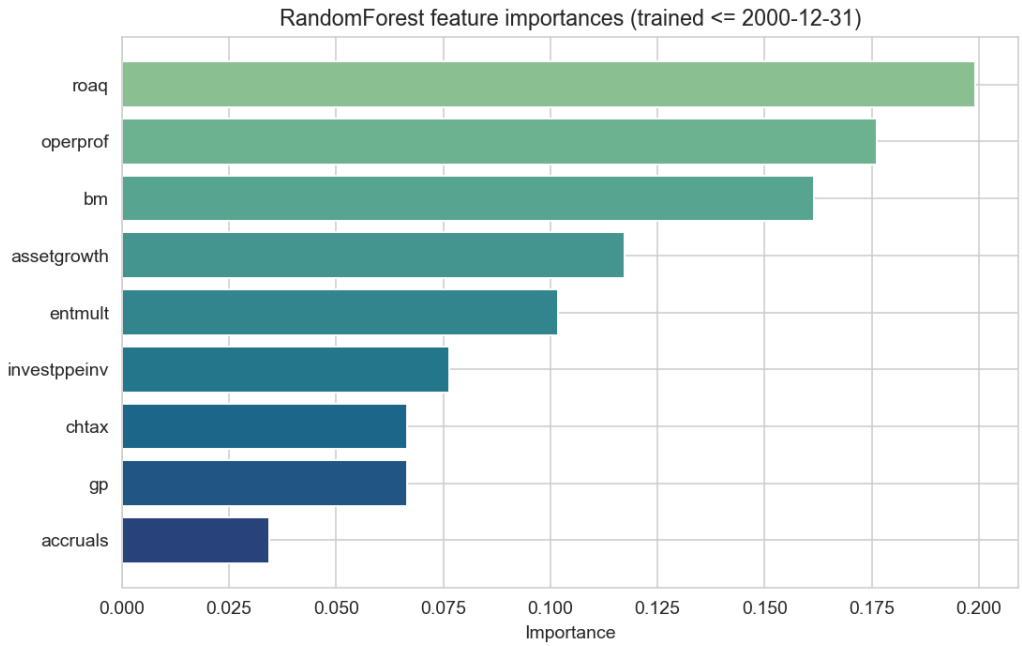

Figure 4: Random Forest feature importances (trained on data ≤ Dec 2000). roaq (quarterly ROA) and OperProf dominate; Accruals contributes least.

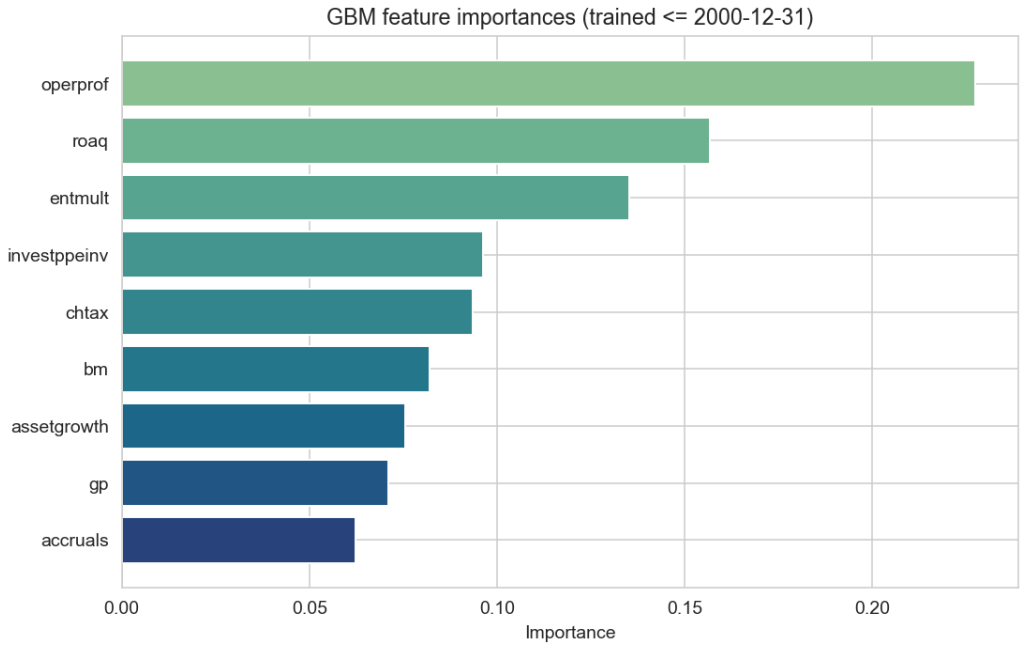

Figure 5: GBM (XGBoost) feature importances. OperProf leads, followed by roaq and EntMult. Signal is more evenly distributed than in Random Forest, reflecting GBM's boosting mechanism.

7. Performance Evaluation - evaluation.py

7.1 Metrics

Metric	Formula	Result range (our run)
Annualised Return	μ_monthly × 12	14.3% – 15.9%
Annualised Sharpe	(μ/σ) × √12	1.019 – 1.084
CAPM Alpha (ann.)	β_0 × 12, from OLS on market return	15.1% – 17.5%
CAPM Beta	β_1, same OLS regression	−0.13 to −0.05
Alpha t-statistic	β_0 / SE(β_0)	5.10 – 5.71
Max Drawdown	min (cum_ret / cum_max_ret − 1)	−22.5% to −28.4%
OOS R^2	$1 - \Sigma(y-\hat{y})^2 / \Sigma y^2$	0.189% – 0.220%

7.2 The GKX OOS R^2 Convention

The OOS R-squared at the firm level conforms to the convention by GKX (2020), where the denominator is the sum of squared realized returns rather than the sum of squared deviations from the mean return. Such a selection is based on the economic motivation that the null forecast for each firm in each period is zero. While our achieved levels of 0.189 – 0.220% may be considered negligible, they are highly competitive with the GKX level of 0.40%, which they achieved using neural networks, using only 9 out of their 94 predictors.

7.3 CAPM Alpha Interpretation

The alpha t-values in all the five cases range from 5.1 to 5.7, indicating high statistical significance regardless of the standard level used. Since the market betas for these portfolios range from –0.05 to –0.13, it clearly shows that the long-short strategies employed are market neutral, meaning that profits have been earned through successful stock-ranking and not exposure to equity market risk.

8. Complexity Analysis - complexity_analysis.py

8.1 Replicating the Virtue of Complexity

The complexity_analysis.py module directly tests the KMZ (2024) claim. Two sweeps are run in parallel:

- Sweep A - Raw predictor count: Ridge is fitted with k ∈ {1, 2, 4, 9} accounting variables. Expected result: OOS R^2 and Sharpe improve as k grows.
- Sweep B - Random Fourier Feature (RFF) expansion: The 9-dimensional input is mapped to a p-dimensional random cosine basis $\varphi(x) = \sqrt{(2/p)} \times \cos(Wx + b)$, sweeping p ∈ {2, 5, 10, 20, 50, 100, 200, 500}. This lets us reach the $p \gg n$ regime without adding new economic variables.

The RFF expansion is key: with only 9 base predictors we cannot observe double-descent by varying the input dimension, but by lifting to p random non-linear features we can observe the KMZ double-descent shape. The projection matrix W is drawn once and held fixed, so it is a deterministic transformation - not a learned parameter - and no look-ahead is introduced.

8.2 Implementation

```
phi(x) = sqrt(2/p) * cos(W @ x + b)    # W ~ N(0,1),  b ~ Uniform[0, 2*pi]
```

Both sweeps reuse the same _ridge_loop() helper that implements the expanding-window walk-forward logic with an optional transform function applied to the feature matrix before fitting. The maximum-width RFF matrix is pre-allocated in a single draw; smaller widths slice the first p rows, ensuring nesting (p=20 sees strictly more information than p=10).

8.3 Expected Results

For sweep A, you should see monotonic improvement; with each added signal, you are getting some incremental cross-sectional informativeness. For sweep B, you should see the double descent curve where R^2 starts improving from p=2 until around p=20, then falls to a minimum near the point of interpolation where p equals the number of firm-month observations per refit, before recovering between p=200 and p=500.

9. Look-Ahead Bias Audit

Look-ahead bias is the single most consequential correctness property of any financial back test. We did a full line-by-line review looking at three classes of failure mode.

9.1 Three Layers of Defence

Layer	Potential failure	Defence implemented
Feature level	Using unannounced accounting data at formation	6-month Fama-French lag: characteristics shifted forward per firm before any normalisation
Cross-section level	Pre-processing pools statistics across time	Median imputation, winsorisation, and rank-normalisation all use groupby('date'); every month is independent
Time-series level	Training target overlaps the test window	Strict cutoff: date < test_start − 1 month; TimeSeriesSplit for inner Ridge CV folds

9.2 Bug 1: Positional fwd_ret Shift (Fixed)

Originally, shift (-1) was employed after groupby(‘permno’) to link the return in the next month. In actual CRSP data, the panel for any firm may have breaks (trading halt, non-coverage period). If there were a break from row t to row t+2, then shift (-1) would incorrectly link the return in row t+2 to row t. This problem is corrected using a dictionary re-indexing function with consideration of dates.

9.3 Bug 2: CAPM Date Misalignment (Fixed)

The returns of the decile portfolios were originally indexed with the date of formation t, while the returns of the market portfolios were indexed with the month in which the realized returns were earned t+1. Hence, in the CAPM regression, the returns for February were regressed on the market returns for January, thus resulting in a biased alpha and beta.

9.4 Items Confirmed Look-Ahead-Free

• Cross-sectional pre-processing: all groupby('date') aggregations calculate statistics based exclusively on that month’s cross-section.
• Walk-forward engine: the training mask employs a stricter ‘less than’ inequality, plus a one-month buffer.
• TimeSeriesSplit: each validation split is later than its respective training split.
• The deciles are determined using y_pred exclusively for the sorting operation, not y_true.
• RFF projection matrix W is constant prior to the walk-forward process and is not modified afterward.
• Importances of features are calculated on the data preceding the train/test cutoff date.

10. Interactive Dashboard - dashboard.py

10.1 Overview

The dashboard is a one-file Streamlit app that makes the entire pipeline available via a web-based UI. The pipeline is configured using the sidebar and clicking on “Run pipeline”; everything from data to models to portfolios to complexity sweep runs within @st.cache_data functions, meaning moving between tabs after the first run takes no time at all, with only the modified stage running if a knob is changed.

10.2 Sidebar Controls

Control	Default	Effect
Start / end year sliders	1990 / 2015	Defines the date window of the simulated panel
Train/test split year slider	2005	Sets the train_end_date for all walk-forward loops
Models multi-select	All five	Enables/disables individual models
Refit frequency	24 months	How often models are retrained (6/12/24/36 options)
Simulated firms	400	Panel size (200/400/600/800 options)
Run complexity sweep checkbox	Enabled	Toggles the RFF complexity analysis on/off
Run pipeline button	-	Clears caches and triggers a full fresh run

10.3 The Four Tabs

Tab 1: Overview

KPI metric strip (best Sharpe, return, alpha, OOS R^2 across all models), styled performance summary table with formatted cells, bar chart for any user-selected metric, cumulative wealth chart on a log-scale Plotly axis, 24-month rolling Sharpe chart, drawdown curves with translucent shaded fill, and a calendar heatmap showing the monthly return pattern of the best-Sharpe strategy year by year.

Tab 2: Portfolios

Decile bar charts for all models’ side by side, a detailed decile breakdown table (mean return, Sharpe, volatility) with an interactive model selector, and feature importance horizontal bar charts for Random Forest and GBM trained on the pre-split window.

Tab 3: Complexity

The two-panel KMZ complexity chart - OOS R^2 vs p on a log-scaled x-axis and Sharpe vs p - for both the raw-predictor sweep (blue circles) and the RFF expansion (orange squares). Raw data tables for both sweeps are shown alongside the chart.

Tab 4: Data

Panel statistics table, feature distribution table (showing that rank-normalisation produces near-uniform distributions), forward-return histogram, firm coverage area chart over time, raw panel preview (first 200 rows), and a CSV download button for the full panel.

10.4 Plotly Bugs Found and Fixed

- Python % string operator clash: hovertemplate strings used the “...%s...” % name syntax, but Plotly’s own template tokens (%{y:.3f} etc.) also use %. Python’s formatter raised TypeError on the unexpected % {sequences. Fixed by replacing all occurrences with string concatenation: “...prefix...” + name + “...suffix...”.
- 8-digit hex colour rejected: Plotly 6 dropped support for #RRGGBBAA hex colours. The drawdown chart used MODEL_COLORS [name] + ‘22’ to encode 13% opacity fill. Fixed by converting to rgba (r, g, b, 0.13) format.

References

Chen, A., & Zimmermann, T. (2022). Open-Source Cross-Sectional Asset Pricing. Critical Finance Review, 11(2), 207–264.

Cooper, M. J., Gulen, H., & Schill, M. J. (2008). Asset Growth and the Cross-Section of Stock Returns. Journal of Finance, 63(4), 1609–1651.

Fama, E. F., & French, K. R. (1992). The Cross-Section of Expected Stock Returns. Journal of Finance, 47(2), 427–465.

Fama, E. F., & French, K. R. (2015). A Five-Factor Asset Pricing Model. Journal of Financial Economics, 116(1), 1–22.

Gu, S., Kelly, B., & Xiu, D. (2020). Empirical Asset Pricing via Machine Learning. Review of Financial Studies, 33(5), 2223–2273.

Haugen, R. A., & Baker, N. L. (1996). Commonality in the Determinants of Expected Stock Returns. Journal of Financial Economics, 41(3), 401–439.

Kelly, B., Malamud, S., & Zhou, K. (2024). The Virtue of Complexity in Return Prediction. Journal of Finance, 79(1), 459–503.

Kelly, B., & Pruitt, S. (2013). Market Expectations in the Cross-Section of Present Values. Journal of Finance, 68(5), 1721–1756.

Lewellen, J. (2015). The Cross-Section of Expected Stock Returns. Critical Finance Review, 4(1), 1–44.

Loughran, T., & Wellman, J. W. (2011). New Evidence on the Relation between the Enterprise Multiple and Average Stock Returns. Journal of Financial and Quantitative Analysis, 46(6), 1629–1650.

Lyandres, E., Sun, L., & Zhang, L. (2008). The New Issues Puzzle: Testing the Investment-Based Explanation. Review of Financial Studies, 21(6), 2825–2855.

Novy-Marx, R. (2013). The Other Side of Value: The Gross Profitability Premium. Journal of Financial Economics, 108(1), 1–28.

Rahimi, A., & Recht, B. (2007). Random Features for Large-Scale Kernel Machines. Advances in Neural Information Processing Systems, 20.

Rosenberg, B., Reid, K., & Lanstein, R. (1985). Persuasive Evidence of Market Inefficiency. Journal of Portfolio Management, 11(3), 9–16.

Sloan, R. G. (1996). Do Stock Prices Fully Reflect Information in Accruals and Cash Flows about Future Earnings? Accounting Review, 71(3), 289–315.

Thomas, J. K., & Zhang, H. (2002). Inventory Changes and Future Returns. Review of Accounting Studies, 7(2–3), 163–187.